

Chapter 6

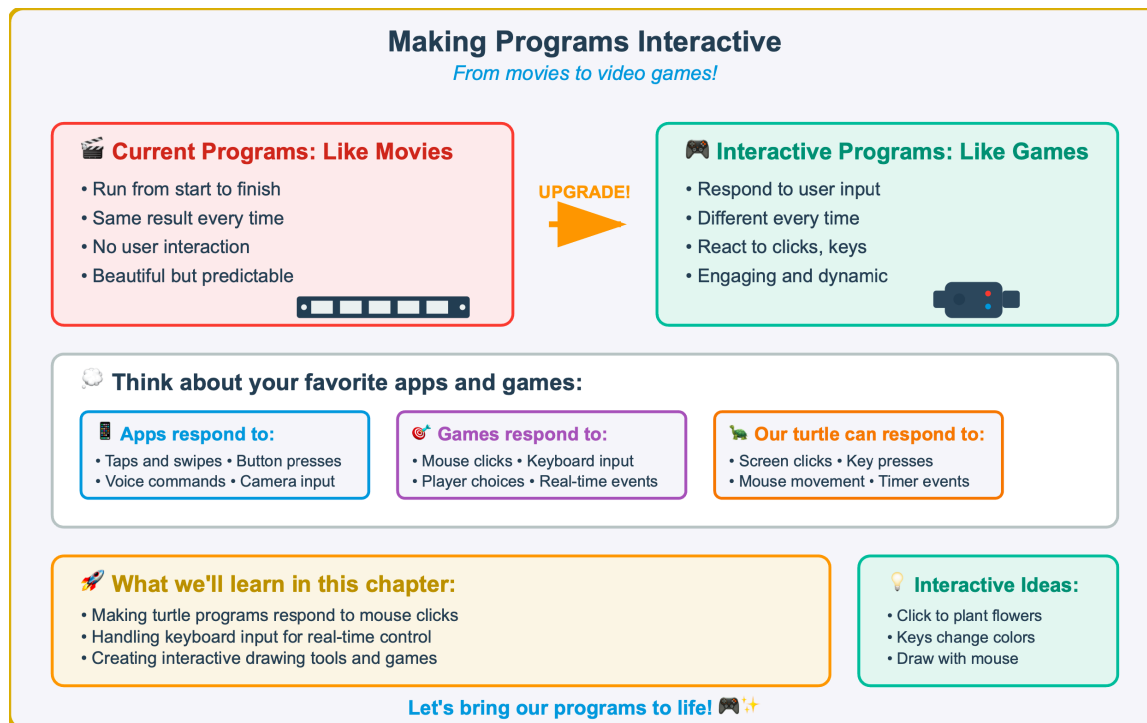


Figure 6.1 — From Static to Interactive Programs — Making Code Responsive. This introduction contrasts traditional linear programs (like movies) with interactive programs (like games) that respond to user input. Students learn how interactive programming transforms predictable code execution into dynamic, user—driven experiences that react to clicks, keyboard input, and real—time events.

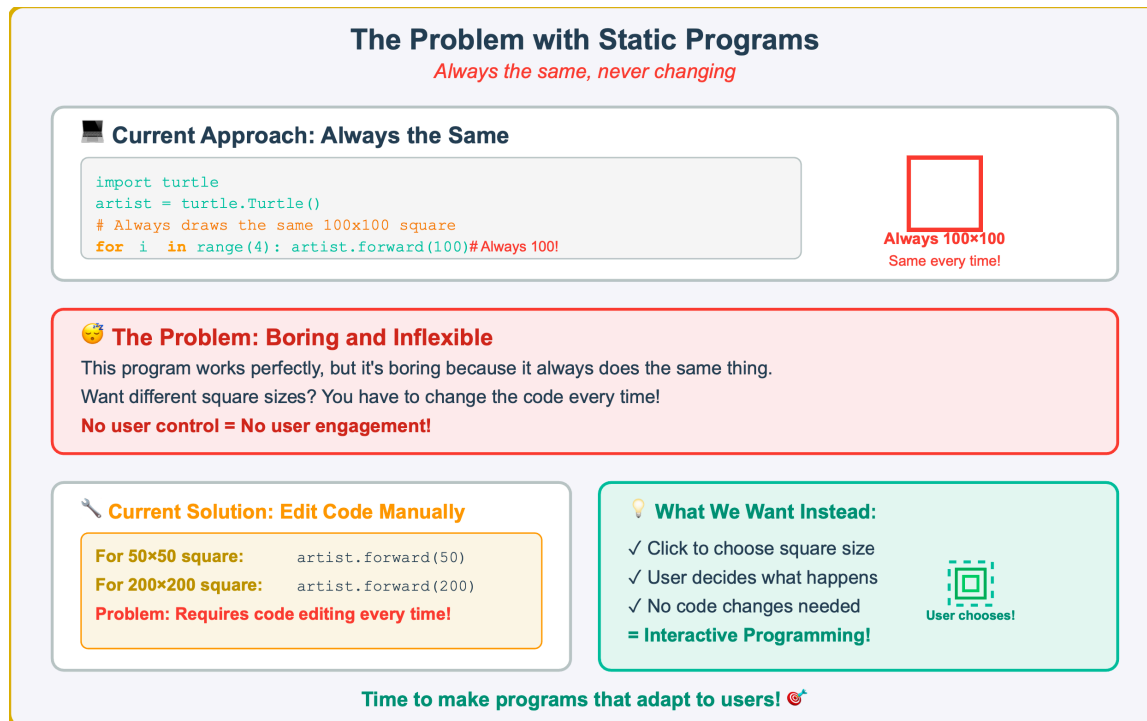


Figure 6.2 — The Limitations of Static Programming — Motivating Interactive Solutions. This example demonstrates why static programs, while functional, are limited by their predictability. The same turtle code always produces identical results, requiring manual code changes for different outcomes. Interactive programming solves this by allowing users to control program behavior through real—time input.

Getting User Input

Making programs respond to user answers

 **The input() Function:**

Ask users questions and get their answers as text - the foundation of interactive programming!

 **Interactive Square Example:**

```
import turtle
artist = turtle.Turtle()

# Ask the user for the square size
size = input("How big should the square be? ")

# Convert text answer to number
size = int(size)

# Now draw with user's chosen size!
for i in range(4): artist.forward(size)
```

 **How It Works:**

1. Program asks question:
"How big should the square be?"

2. User types answer:
"150" (text entered by user)

3. Program converts uses:
int("150") → 150 (number)

 User controls the program!

 **Important to Remember:**

- input() always returns text (string), even if user types numbers
- Use int() to convert text numbers to actual numbers for math
- Always include clear, friendly prompts in your questions

 **The Result:**

Users control square size
No code changes needed
Interactive programming!

Users are now in control - your programs can adapt! 🤖

Figure 6.3 — Getting User Input — The input() Function in Action. This example demonstrates how the input() function enables basic user interaction by prompting users for information and capturing their responses as text. Students learn the essential pattern of asking questions, receiving answers, and converting text input to appropriate data types for program use.

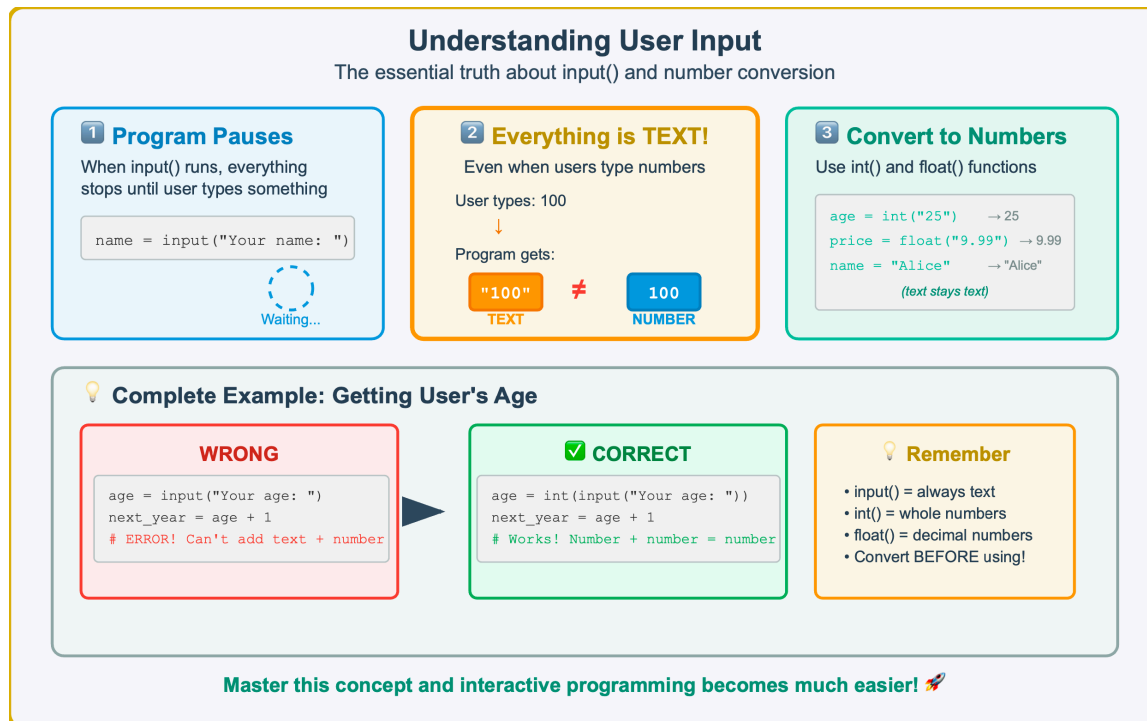


Figure 6.4 — Understanding User Input — From Text to Numbers. This diagram reveals the crucial fact that input() always returns text, even when users type numbers. Understanding this concept and knowing how to convert text to numbers is fundamental to creating interactive programs.

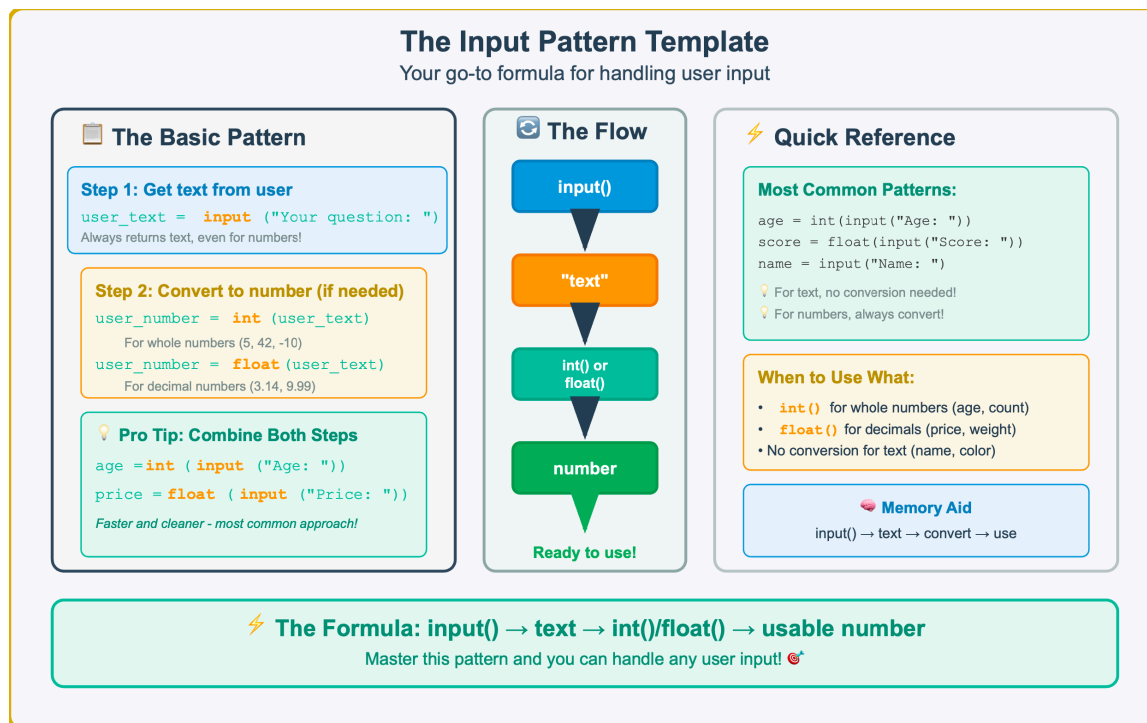


Figure 6.5 — The Input Pattern Template — Your Go—To Formula. This template provides the essential pattern for getting user input and converting it to usable numbers. Master this formula and you can handle any input situation in your programs.

Custom Polygon Challenge

🚀 Putting User Input to Work

🚀 The Challenge

Create a program that asks the user how many sides they want, then draws that exact polygon!

💡 The Secret

All polygons turn 360° total

$$\text{angle} = 360 \div \text{sides}$$

Triangle: $360 \div 3 = 120^\circ$
Square: $360 \div 4 = 90^\circ$

🛠 Skills Applied

- ✓ User input with `int()`
- ✓ Mathematical calculation
- ✓ Loop with variable range

🎨 One Program, Infinite Possibilities!

🖥 Complete Solution

```
import turtle
screen = turtle.Screen()
artist = turtle.Turtle()

# Get user input
sides = int(input("How many sides? "))

# Calculate the angle
angle = 360 / sides

# Draw the polygon
for i in range(sides):
    artist.forward(100)
    artist.right(angle)

# Keep window open
screen.exitonclick()
```

🔍 Notice the input pattern!

🎨 What You Can Create



3 sides
Triangle



4 sides
Square



5 sides
Pentagon



6 sides
Hexagon



8 sides
Octagon

📐 The Math

- More sides = smaller angles
- All polygons total 360°

🚀 Try: 7, 12, or 20 sides!

💡 This is the power of programming: Write once, create infinitely! 💡

Figure 6.6 — Custom Polygon Challenge — Putting Input to Work. This exercise demonstrates how user input transforms static programs into interactive experiences. By asking users how many sides they want, a single program can create any polygon, showcasing the power of combining input handling with mathematical concepts.

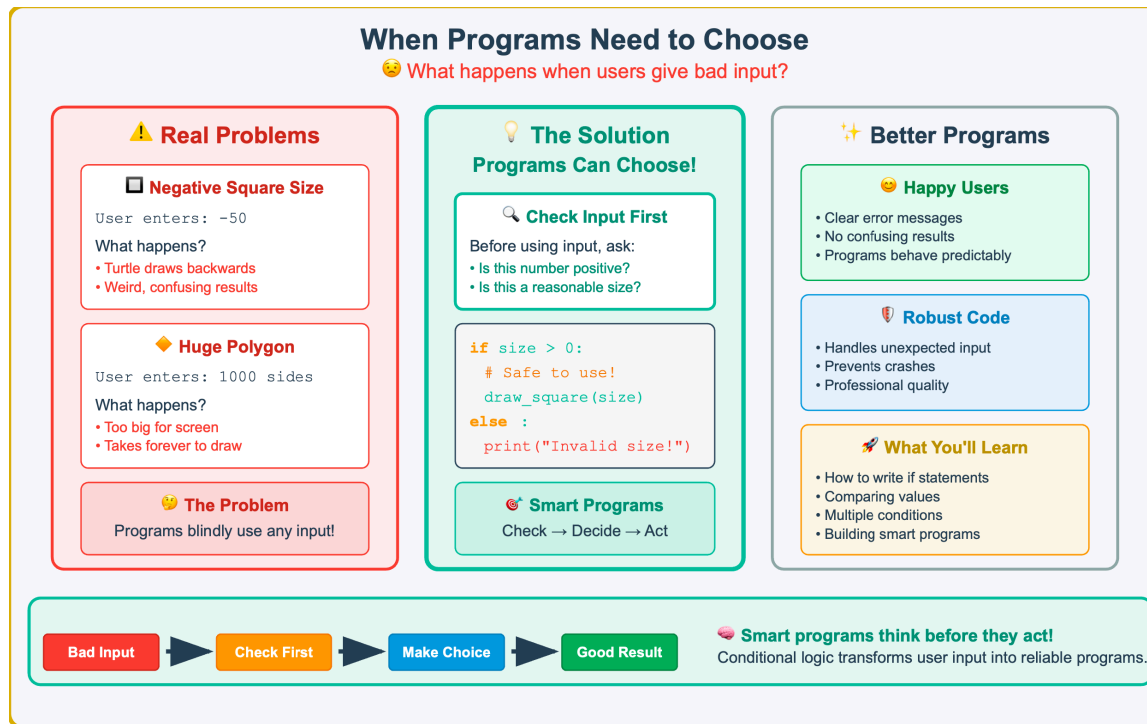


Figure 6.7 — When Programs Need to Choose — Introduction to Decision Making. This figure demonstrates why programs need conditional logic by showing real problems that occur with invalid user input. Smart programs check input values and make appropriate decisions, leading to better user experiences and more robust code.

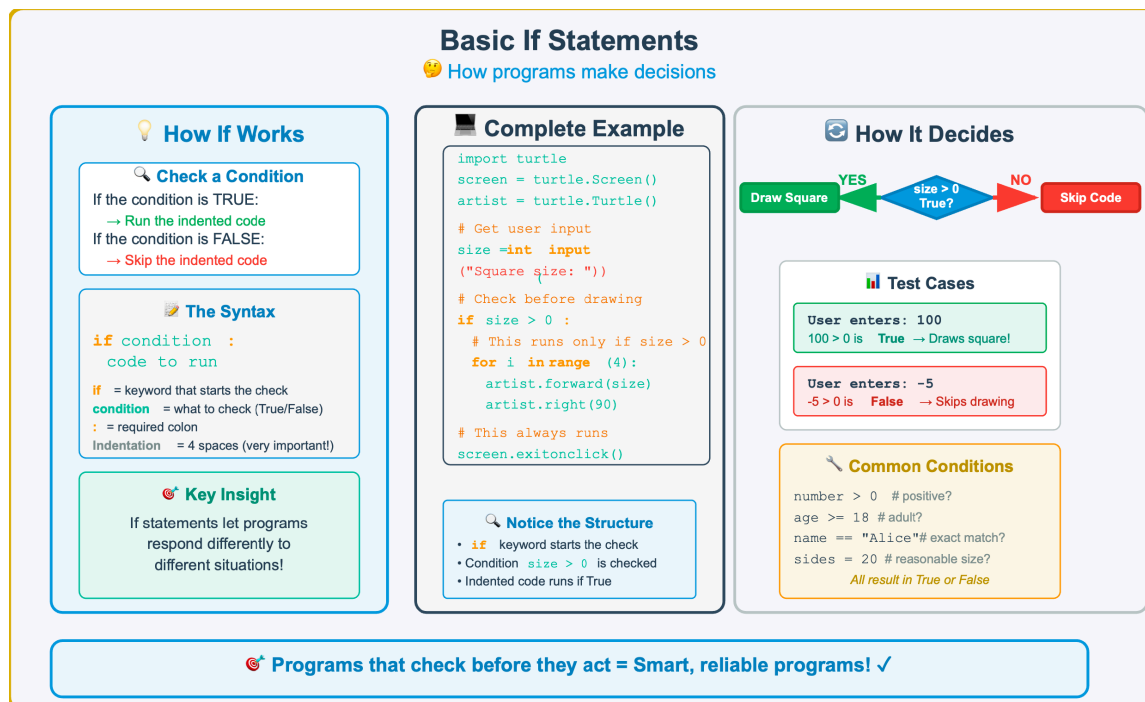


Figure 6.8 — Basic If Statements — How Programs Make Decisions. This diagram shows the fundamental structure of if statements: check a condition, and only execute code if the condition is true. This simple concept is the foundation of all program decision—making and transforms static code into intelligent, responsive programs.

Comparison Operators

🔍 Building conditions for if statements

📁 All Six Operators

EQUALITY TESTING

== equal to
size == 10, name == "Alice"

!= not equal to
color != "red", answer != "yes"

SIZE COMPARISON

> greater than
age > 18, speed > 60

< less than
sides < 20, price < 100

RANGE TESTING

>= greater or equal
score >= 90

<= less or equal
sides <= 12

📄 Examples in Action

```
if size > 0:
    draw_square()

if color == "red":
    turtle.color("red")

if sides >= 3:
    draw_polygon()

if answer != "quit":
    keep_playing()

if speed < 60:
    print("Safe speed")

if attempts <= 3:
    try_again()
```

Each returns True or False

💡 Usage Guide

🔥 Common Uses

- Validate user input (size > 0)
- Check game conditions (score >= 100)
- Menu choices (answer == "quit")
- Safety checks (speed = limit)
- Age verification (age >= 18)

⚠️ Important!

== for comparing (equal to)
= for assigning (give value)
Don't mix these up!

📖 Quick Reference

All comparisons return:

- **True** if the comparison is correct
- **False** if the comparison is wrong

🔍 Compare values to make smart decisions - the foundation of intelligent programs!

Figure 6.9 — Comparison Operators — Building Conditions for If Statements. This reference guide shows the six comparison operators that create conditions for if statements. Each operator compares two values and returns True or False, enabling programs to make intelligent decisions based on data comparisons.

Safe Polygon Challenge

Combining conditions with "and"

The Challenge

Your Mission
Improve the polygon program!
Only draw if user enters a reasonable number of sides.

The Key: "and"
Need to check TWO things:

- At least 3 sides (sides ≥ 3)
- No more than 20 sides (sides ≤ 20)

`sides  $\geq 3$  and sides  $\leq 20$`

Valid Range

Complete Solution

```
import turtle
screen = turtle.Screen()
artist = turtle.Turtle()

# Get user input
sides = int(input("How many sides?"))

# Check valid range
if sides >= 3 and sides <= 20:
    # Safe to draw!
    angle = 360 / sides
    for i in range(sides):
        artist.forward(50)
        artist.right(angle)

screen.exitonclick() # Keep window open
```

Both Conditions Must Be True
`sides  $\geq 3$  AND sides  $\leq 20$`

Test Cases

Valid Input (Draws Polygon)

User enters: 5
5 ≥ 3 ? $\checkmark$ 5 ≤ 20 ? $\checkmark$ → Draws pentagon!

User enters: 12
12 ≥ 3 ? $\checkmark$ 12 ≤ 20 ? $\checkmark$ → Draws 12-gon!

Invalid Input (Skips Drawing)

User enters: 2
2 ≥ 3 ? X → Skips (too few sides)

User enters: 50
50 ≤ 20 ? X → Skips (too many sides)

User enters: -3
-3 ≥ 3 ? X → Skips (negative)

How "and" Works

Both conditions must be True
If either is False → skip the code

Smart programs check ranges - preventing problems before they happen!

Figure 6.10 — Safe Polygon Challenge — Combining Conditions with ‘and’. This exercise demonstrates how to use compound conditions to validate input ranges. By combining two conditions with ‘and’, the program ensures that user input falls within acceptable bounds, creating more robust and user—friendly programs.

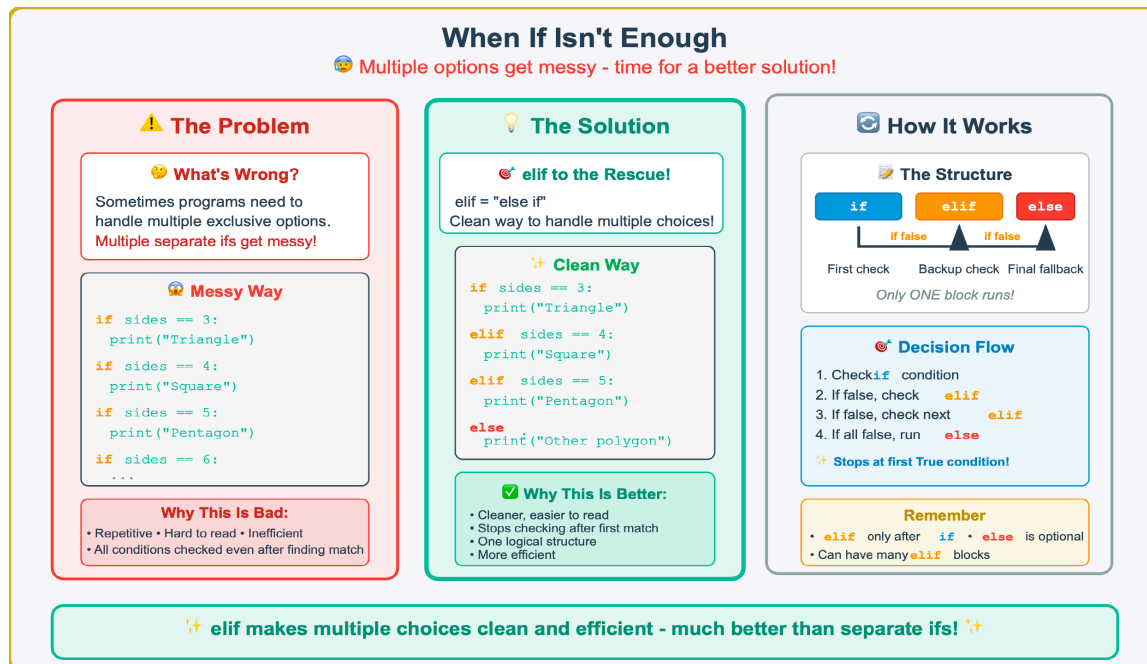


Figure 6.11 — When If Isn't Enough — Introducing elif for Multiple Choices. This diagram shows why multiple separate if statements become problematic and introduces elif as the clean solution. The elif statement (short for “else if”) provides an organized way to handle multiple mutually exclusive conditions in a single logical structure.

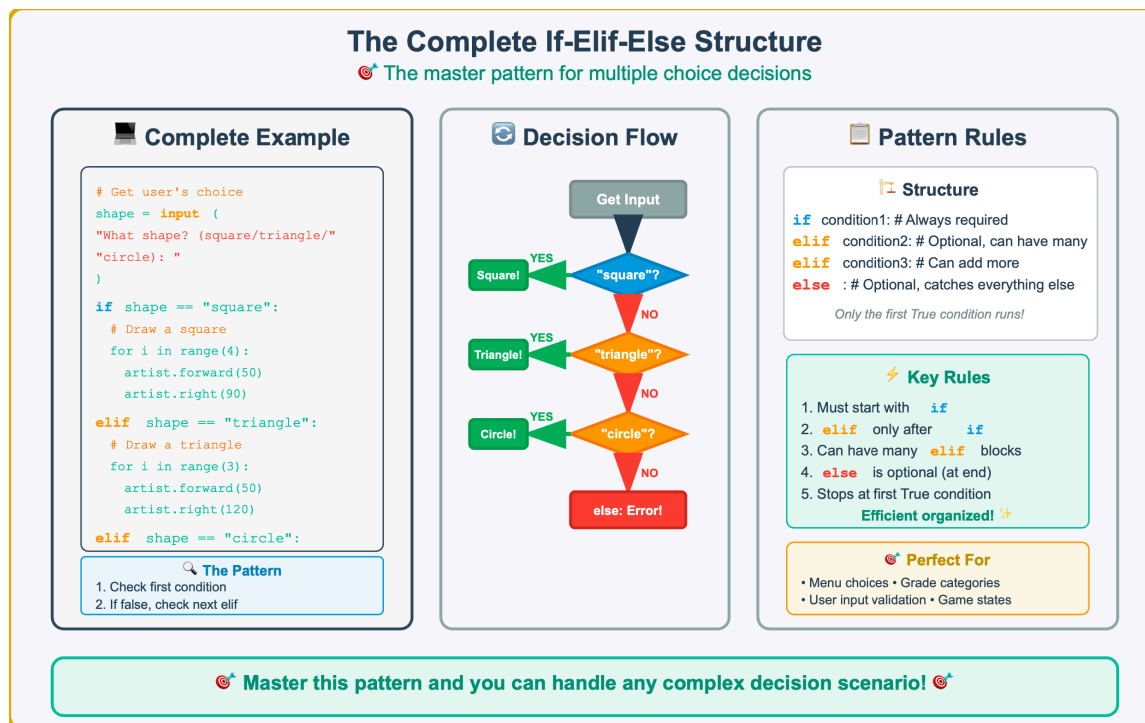


Figure 6.12 — The Complete If—Elif—Else Structure — Multiple Choice Decision Making. This diagram shows the full if—elif—else pattern in action, demonstrating how programs can handle multiple exclusive options efficiently. The structure checks conditions sequentially and executes only the first matching block, providing a clean and organized approach to complex decision—making.

Color Chooser Challenge

🎨 If-elif-else in creative action!

🎯 Your Challenge

🚀 **Make Art Interactive!**

Let users choose the color for their drawings using the **if-elif-else pattern!**

⚡ **Perfect Pattern Use**

- ✓ Multiple exclusive choices
- ✓ Each option does different thing
- ✓ Safe default with else
- ✓ User-friendly experience

This is what if-elif-else was made for!

🌟 **Why This Rocks**

- Programs become personal
- Users feel in control
- Creativity meets code

Programming = artistic tool! 🎨

💻 Complete Solution

```
import turtle
screen = turtle.Screen()
artist = turtle.Turtle()

# Get user's color choice
color = input (
    "Pick a color (red/blue/"
    "green/purple): "
)

if color == "red":
    artist.color("red")
elif color == "blue":
    artist.color("blue")
# ... green, purple ...

else :
    artist.color("black") # Safe!
```

💡 **Smart Default!**

else handles typos and unexpected input safely - program never crashes!

🎨 Color Palette

Available Choices

red

blue

green

purple

default

👤 User Experience

User types: "blue"

→ Gets beautiful blue drawings! ✨

User types: "pink"

→ Gets black (safe default) 🛡️

Program never crashes or fails!

🚀 **Level Up Ideas**

- Add more colors • Shape choices
- Size options • Pattern selection

🎨 You've mastered conditional logic - now make programming personal and creative! 🎨

Figure 6.13 — Color Chooser Challenge — If—Elif—Else in Creative Action. This exercise demonstrates the if—elif—else pattern in a creative context, allowing users to customize their artwork through color selection. The else clause provides a safe default, showcasing how robust programs handle unexpected input gracefully while maintaining functionality.

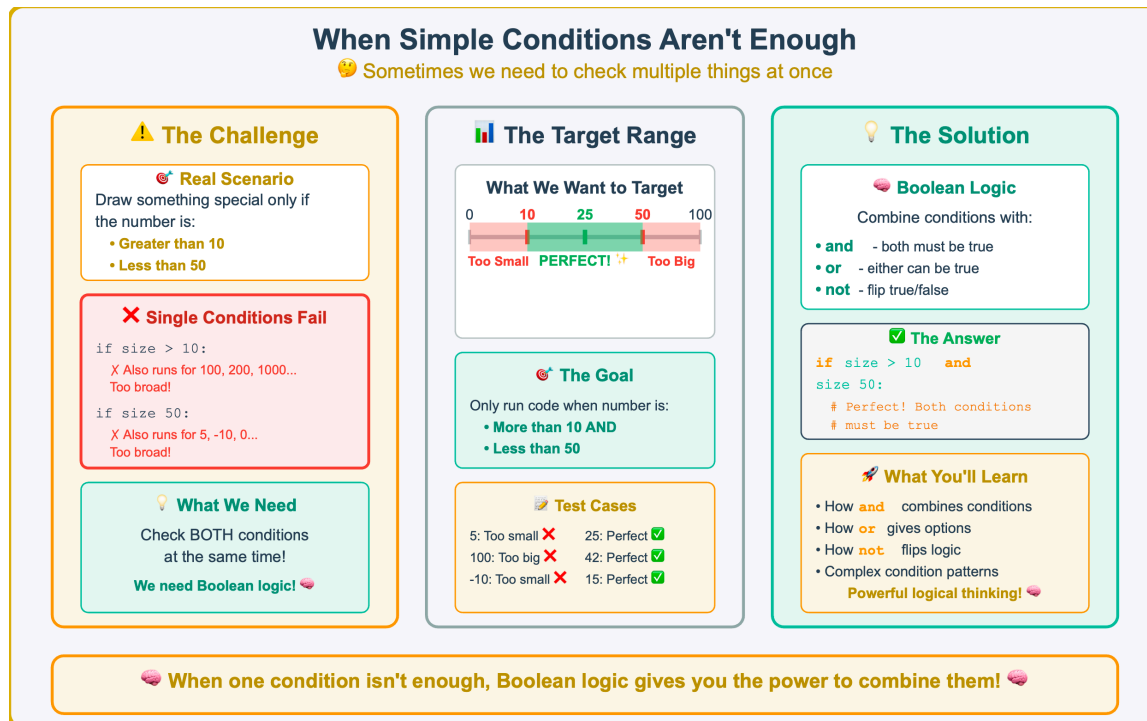


Figure 6.14 — When Simple Conditions Aren't Enough — The Need for Boolean Logic. This diagram illustrates scenarios where single conditions are inadequate for complex decision-making. When programs need to check multiple criteria simultaneously, such as validating ranges or combining requirements, Boolean logic operators become essential for creating precise, reliable conditions.

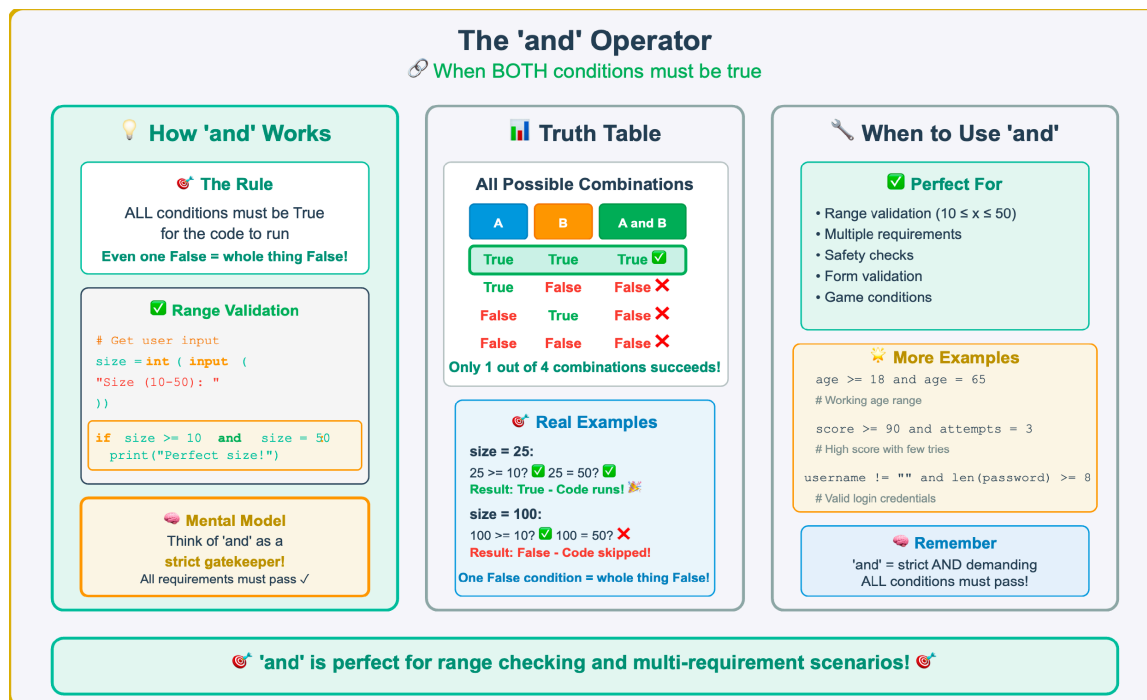


Figure 6.15 — The 'and' Operator — When Both Conditions Must Be True. This diagram explains how the 'and' operator combines multiple conditions, requiring all conditions to be true for the code block to execute. The truth table demonstrates all possible combinations, showing that even one false condition makes the entire expression false, making 'and' perfect for range validation and multi—requirement scenarios.

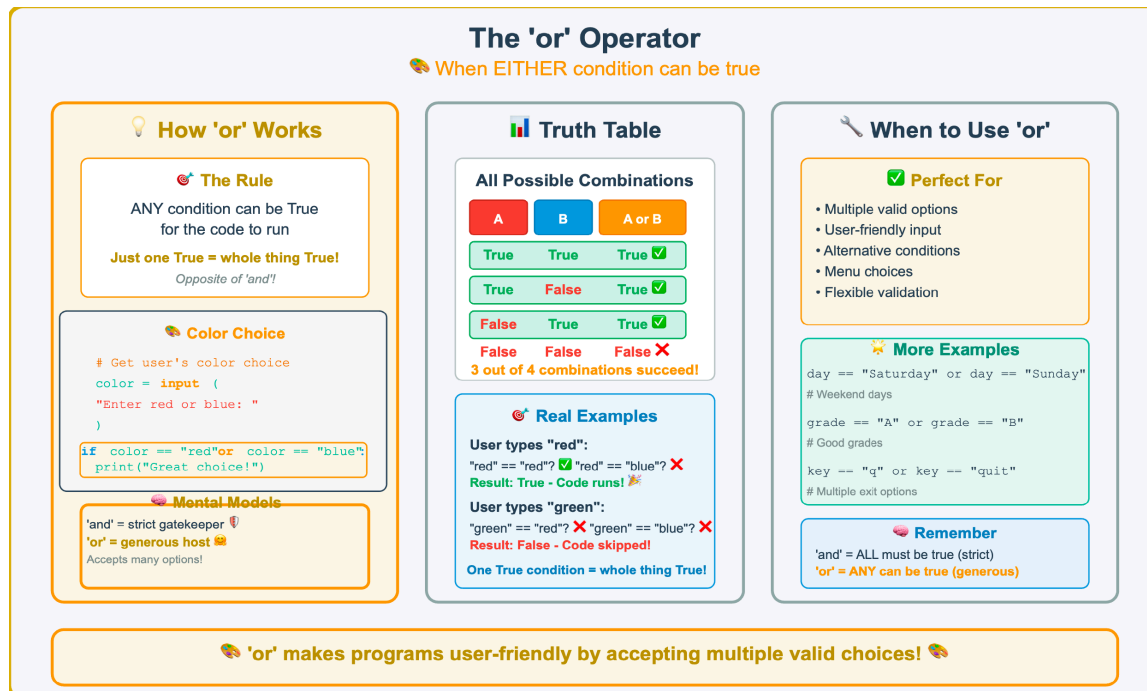


Figure 6.16 — The 'or' Operator — When Either Condition Can Be True. This diagram explains how the 'or' operator provides flexible conditional logic by executing code when any condition is true. Unlike 'and' which requires all conditions to be true, 'or' succeeds when at least one condition is met, making it perfect for offering users multiple valid choices and options.

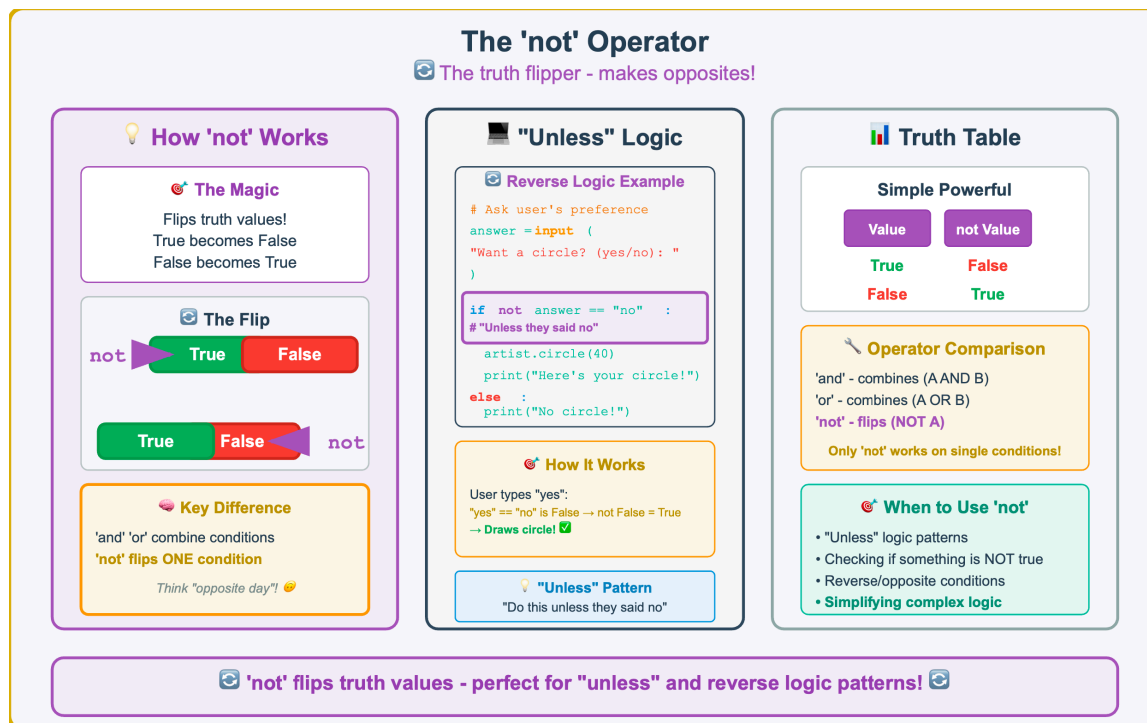


Figure 6.17 — The 'not' Operator — Flipping Truth Values. This diagram explains how the 'not' operator reverses logical values, transforming True into False and False into True. Unlike 'and' and 'or' which combine conditions, 'not' operates on a single condition to create its logical opposite, enabling "unless" and reverse logic patterns in programming.

Try It Yourself! Smart Drawing Program

Complex Conditional Logic Exercise

The Challenge

Create a program that only draws if ALL conditions are met!

Size Requirements

- Must be between 10 and 100
- (size >= 10 AND size = 100)

size >= 10 and size = 100

Color Requirements

- Must be red OR blue OR green
- (exactly one of these colors)

color in ["red", "blue", "green"]

Both Must Be True

(Size check) AND (Color check)

If either fails → no drawing!

Complete Code Solution

```
1 import turtle
2 artist = turtle.Turtle()
3
4 # Get user input
5 size = int(input("Size: "))
6 color = input("Color: ")
7
8 # Complex condition check
9 if (size >= 10 and size = 100) and
10 (color == "red" or color == "blue"
11 or color == "green"):
12     artist.color(color)
13 # Draw the square
14 for i in range(4):
15     artist.forward(size)
16     artist.right(90)
```

Logic Analysis

Condition Structure:

(Size Check) AND (Color Check)

Both must be TRUE

Test Cases

PASS Examples:

size=50, color="red"

size=25, color="blue"

size=100, color="green"

FAIL Examples:

size=5, color="red" (size too small)

size=50, color="yellow" (invalid color)

Key Learning:

- AND: ALL conditions must be true
- OR: ANY condition can be true

Figure 6.18 — Smart Drawing Program — Complex Conditional Logic Exercise. This hands—on challenge teaches students to combine multiple conditions using AND and OR operators, demonstrating how real programs make complex decisions based on multiple criteria.

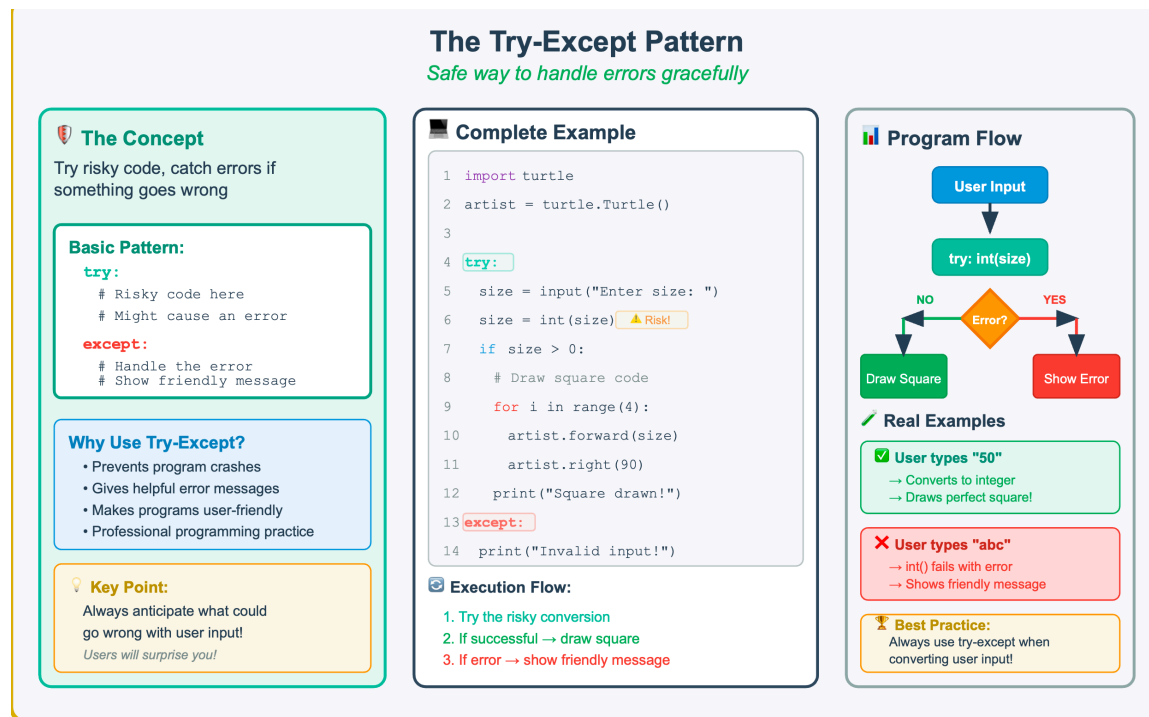


Figure 6.19 — The Try—Except Pattern — Safe Error Handling. This essential programming pattern teaches students how to anticipate and gracefully handle errors, preventing programs from crashing and providing helpful feedback to users.

Try It Yourself! Bulletproof Input

Advanced Input Validation Exercise

The Challenge

Keep asking for valid color until user enters one!

No matter what they type, handle it gracefully!

While Not Loop

- Keeps running until condition is True
- "not False" = True (keep going)
- "not True" = False (stop loop)

```
while not valid_input:
```

"In" Operator

- Checks if item exists in list
- "red" in ["red", "blue"] → True
- "pink" in ["red", "blue"] → False

```
if color in valid_colors:
```

The Strategy

- Start with valid_input = False
- Loop while NOT valid
- Check input against allowed list
- Set valid_input = True to exit

Complete Solution

```
1 import turtle
2 artist = turtle.Turtle()
3
4 # Setup valid colors list
5 valid_colors = ["red", "blue",
6 "green", "yellow"]
7 valid_input = False # Flag
8
9 # Keep asking until valid
10 while not valid_input:
11     color = input("Choose color: ")
12     if color in valid_colors:
13         valid_input = True # Exit!
14     else:
15         print("Try again!") # Loop!
```

Program Flow:

- Start loop (valid_input = False)
- Check if input is in allowed list
- If yes → exit loop, if no → ask again

Validation Flow

```
graph TD
    Start([valid_input = False]) --> Loop{while not valid_input}
    Loop --> Input[Get user input]
    Input --> Check{color in valid_colors?}
    Check -- NO --> Error[Show error]
    Check -- YES --> Exit([Exit loop!])
    Error --> Loop
    Exit --> End([End])
```

Example Interaction

User Interaction Timeline:

```
Choose color:"pink"
X Try again!
Choose color:"orange"
X Try again!
Choose color:"blue"
✓ Perfect! Continue...
```

Benefits:

- Never crashes on bad input
- Guides user to valid choices

Figure 6.20 — Bulletproof Input Exercise — Building Robust User Input Systems. This comprehensive challenge combines while loops, list validation, and user—friendly error handling to create programs that persistently guide users toward valid input, demonstrating professional—grade input validation techniques.

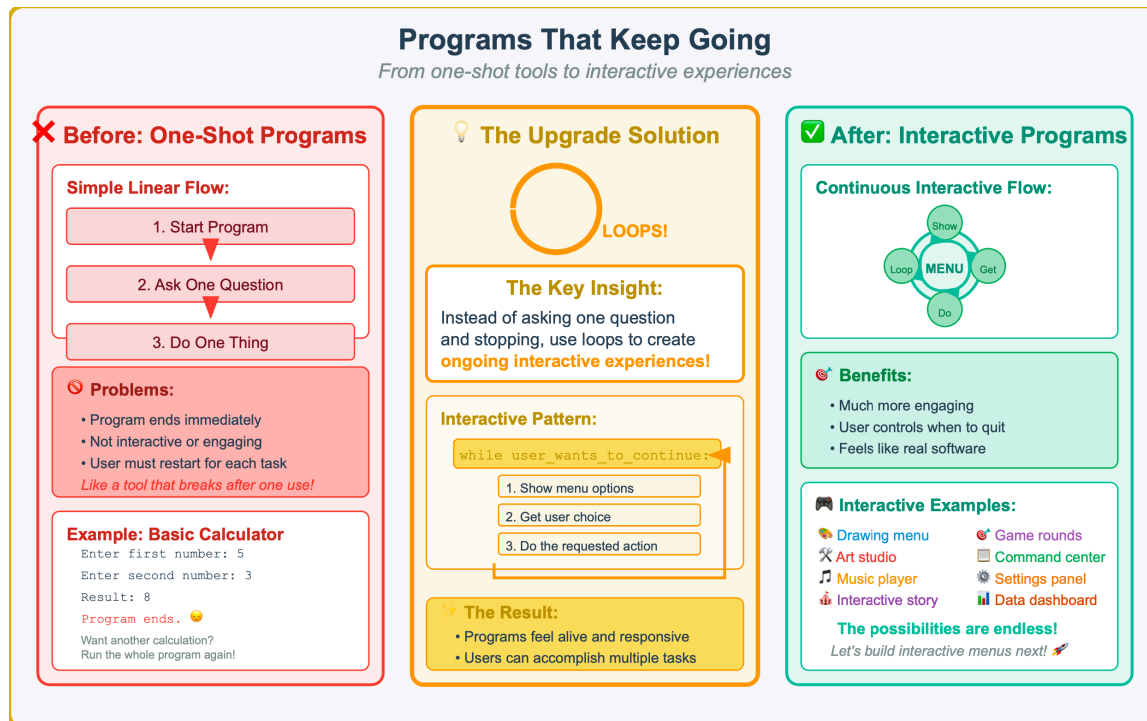


Figure 6.21 — Programs That Keep Going — From One—Shot to Interactive Applications. This conceptual framework introduces students to the mindset shift from simple, single—purpose programs to engaging, interactive applications that continue running until users choose to exit, demonstrating the foundation of modern software design.

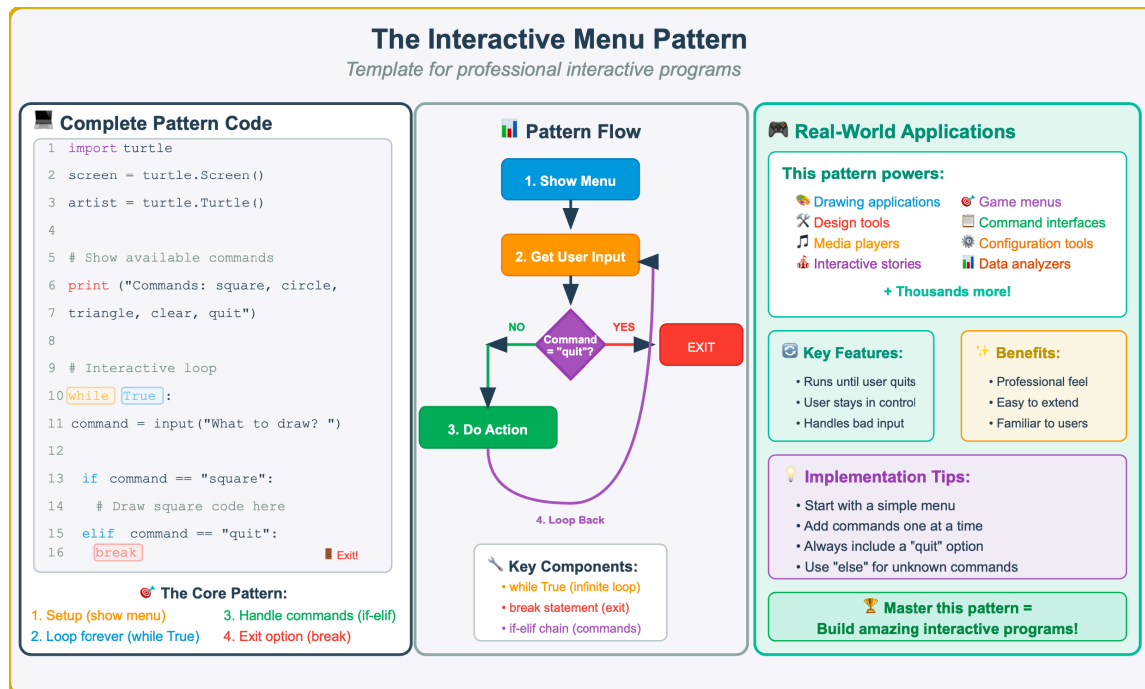


Figure 6.22 — The Interactive Menu Pattern — Template for Professional Interactive Programs. This essential programming pattern combines while loops, conditional statements, and user input to create the foundation of interactive software. Students will use this template repeatedly to build everything from simple drawing tools to complex applications.

Try It Yourself! Simple Art Studio

Capstone Project: Build Your Creative Interactive Application

The Challenge

Build an interactive art studio with at least 5 different drawing options!

Code Structure:

```
# 1. Define drawing functions
def draw_flower():
    for i in range(6):
        artist.circle(20)
        artist.right(60)

# 2. Show menu
print("Options: flower, star...")

# 3. Interactive loop
while True:
    choice = input("What to draw?")
    if choice == "flower":
```

Concepts You'll Use:

- ✓ Functions (def) - organize drawing code
- ✓ While loops - keep program running
- ✓ If-elif chains - handle user choices
- ✓ Interactive menus - professional feel

= All of Chapter 6 in one project!

Required Features

Drawing Options (5 minimum):

- ✖ 1. flower (6-petal pattern)
- + 2. star (5-pointed shape)
- 3. circle (simple shape)
- 4. random color changer
- 5. your creative choice!
(triangle, spiral, heart, etc.)

Control Options:

- clear - erase the canvas
- quit - exit the program
- error handling for bad input

Success Criteria:

- Program runs continuously
- At least 5 drawing options work
- Clear and quit functions work
- Handles unknown commands
- Your personal creative touch!

Visual Examples Ideas

What your art studio can create:

Flower
Star

Creative Ideas!

Extension Ideas:

- Ask user for size/color preferences
- Add position control (move turtle)
- Create pattern sequences
- Add speed controls or animations

Development Tips:

1. Start with one drawing function
2. Test the menu loop early
3. Add features one at a time

Create your own interactive art studio! 🎨🔧

Figure 6.23 — Simple Art Studio Exercise — Building Your Creative Interactive Application. This comprehensive capstone project integrates all Chapter 6 concepts, functions, loops, conditionals, and interactive menus, into a creative art studio that demonstrates how programming skills combine to create engaging, user—controlled applications.

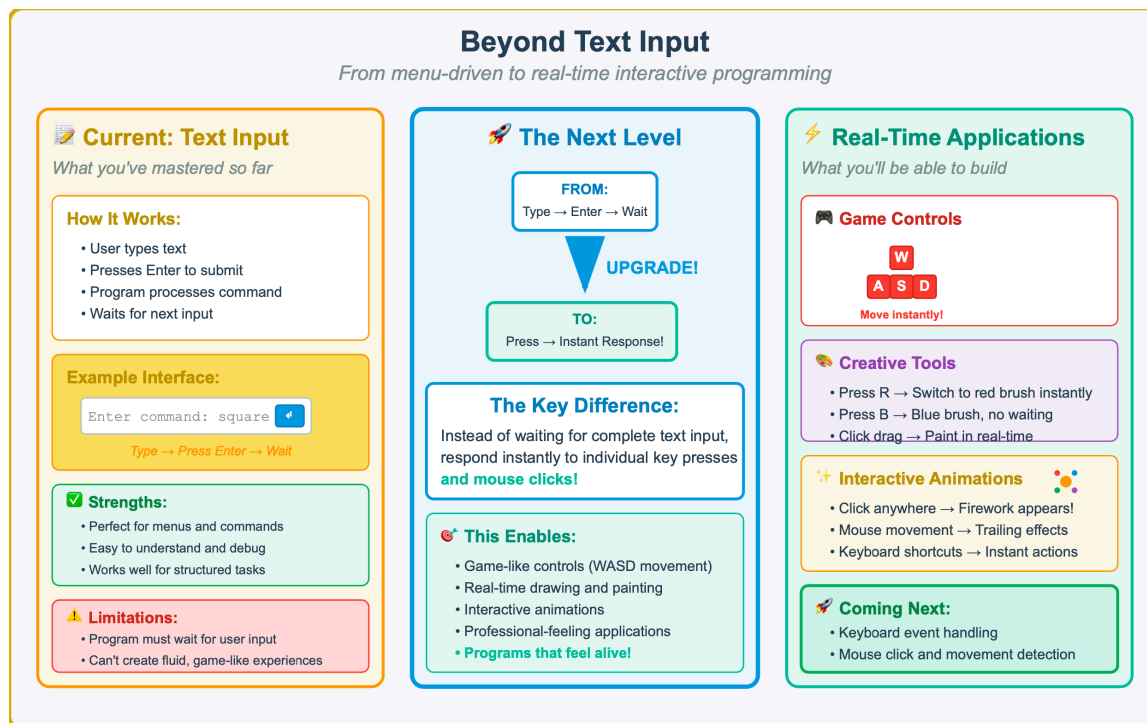


Figure 6.24 — Beyond Text Input — Transition to Real—Time Interactive Programming. This forward—looking concept introduces students to the next evolution in interactive programming: moving from menu—driven text input to instant, real—time responses that create game—like experiences and professional interactive applications.

Turtle Key Control

Implementing real-time keyboard input

Complete Implementation

```
1 import turtle
2
3 # Setup screen and turtle
4 screen = turtle.Screen()
5 player = turtle.Turtle()
6 player.shape("turtle")
7
8 # Movement functions
9 def move_up():
10     player.setheading(90)
11     player.forward(20)
12
13 # Similar functions for down, left, right
14
15 # Bind keys to functions
16 screen.onkey(move_up, "Up")
17 screen.onkey(move_down, "Down")
18 screen.listen() 🔑 Key!
```

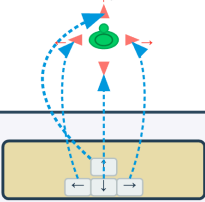
Key Concepts:

- `screen.onkey()` - binds keys to functions
- `screen.listen()` - enables key detection
- Functions called instantly on key press

Interactive Demo

Turtle Graphics

Use arrow keys to move!



How It Works:

1. Press arrow key
2. Function runs instantly
3. Turtle moves immediately

No More Waiting!

- No typing commands
- No pressing Enter

Extensions Ideas

More Key Bindings:

- Space bar → Change color
- R key → Reset position
- P key → Pen up/down
- + / - keys → Change speed

Game Ideas:

- Maze navigation game
- Collect items challenge
- Drawing race patterns

Setup Requirements:

1. Create movement functions first
2. Bind each key with `screen.onkey()`
3. Don't forget `screen.listen()`!

Congratulations!

You've built your first real-time interactive program!
This is just the beginning... 🚀

Figure 6.25 — Turtle Key Control — Implementing Real—Time Keyboard Input. This practical demonstration shows how to implement real—time keyboard control, allowing users to move a turtle instantly using arrow keys without typing commands or pressing Enter. This represents the first step into event—driven programming.

Click to Control

Implementing real-time mouse input

Complete Implementation

```

1 import turtle
2
3 # Setup screen and turtle
4 screen = turtle.Screen()
5 artist = turtle.Turtle()
6
7 # Click handler function
8 def goto_click(x, y):
9     artist.goto(x, y)
10
11 # Bind click to function
12 screen.onclick(goto_click)
13 screen.listen()

```

Coordinate System:

- Click gives you (x, y) coordinates
- Center of screen is (0, 0)
- goto(x, y) moves turtle to that spot

Key Concepts:

- Function receives x, y parameters automatically
- screen.onclick() binds function to clicks
- Each click calls function with click location

Interactive Demo

Turtle Graphics

Click anywhere to move turtle!

How It Works:

- User clicks anywhere on screen
- Function receives (x, y) coordinates
- Turtle moves to that location
- Ready for next click!

Mouse Magic:

- Click = instant movement
- No typing, no Enter key needed

vs. Text Input:

Text: "goto 100 50" + Enter
Click: Just click where you want!

Advanced Applications

Combining Controls:

- Arrow keys + mouse clicks
- Keyboard shortcuts + point-and-click

= Complete control system!

Game Applications:

- Point-and-click adventures
- Strategy games (click to move units)
- Drawing and art programs

Enhanced Functions:

- Different actions for different areas
- Change colors based on click location
- Draw trails as turtle moves

Professional Uses:

- CAD software (click to place objects)
- Map applications (click destinations)

You're learning real industry skills!

Figure 6.26 — Click to Control — Implementing Real—Time Mouse Input. This practical demonstration shows how to implement mouse click handling, allowing users to move a turtle instantly to any clicked location on the screen. Combined with keyboard control, this completes the foundation of modern interactive programming.

Interactive Drawing Application

Combining keyboard movement and mouse drawing

Complete Implementation

```

1 import turtle
2 screen = turtle.Screen()
3 artist = turtle.Turtle()
4
5 # Movement functions
6 def move_up():
7     artist.setheading(90)
8     artist.forward(10)
9
10 # Similar for down, left, right
11
12 # Drawing function
13 def draw_dot(x, y):
14     artist.goto(x, y)
15     artist.dot(8, "red")
16

```

Input System Setup:

```

screen.onkey(move_up, "Up")
screen.onclick(draw_dot)

```

Integration Pattern:

- Keyboard: Precise movement control
- Mouse: Creative drawing anywhere

Interactive Demo

Creative Drawing Studio

Keyboard Controls:

- Arrow keys move turtle precisely
- 10 pixel increments
- Perfect for precise positioning

Mouse Controls:

- Click anywhere to draw dot
- Instant creative expression
- Freedom to draw anywhere

Best of Both!

Precision + Creativity

Creative Extensions

Enhanced Features:

- Number keys change colors
- Space bar changes pen size
- C key clears the canvas
- S key saves artwork!

Professional Uses:

- Digital art applications
- CAD design tools
- Interactive whiteboards

This is real industry code!

Game Development:

- Character movement + interaction
- Building and crafting games

You Did It!

You've built a complete interactive application!

Professional programmer level! 🚀

Figure 6.27 — Interactive Drawing Application — Combining Keyboard and Mouse Controls. This comprehensive interactive application demonstrates how to integrate both keyboard movement controls and mouse click drawing into a single creative tool. This synthesis represents the culmination of real—time interactive programming concepts and shows how professional creative applications are structured.

Number Guessing Game Challenge

Capstone programming project - Apply all your skills!

Complete Game Implementation

```

1 import random
2 import turtle
3
4 # Game setup
5 secret = random.randint(1, 10)
6 guesses_left = 3
7 won = False
8
9 # Main game loop
10 while guesses_left > 0 and not won:
11     try:
12         guess = int(input("Guess: "))
13         if guess == secret:
14             won = True
15         elif guess < secret:
16             print("Too low!")
17         except:
18             print("Invalid input!")

```

Skills You're Using:

- Random numbers • While loops • Try-except
- Conditionals • Input validation • Game logic

Game Experience

Number Guessing Game

- 1 Computer picks secret (1-10)
- 2 You have 3 guesses
- 3 Get feedback: too high/low
- 4 Win = Turtle celebration!

Example Game:

Guess: 5
Too low!
Guess: 8
Too high!

🎉 You Won!

✨

Game Features:

- Smart feedback • Error handling
- Visual celebration • Replay option

Game Development Path

Game Extensions:

- Difficulty levels (1-100, 1-1000)
- Score tracking and high scores
- Multiplayer turns
- Graphical interface with buttons

Professional Concepts:

- Game state management
- User experience design
- Error handling and edge cases

You're thinking like a game developer!

More Game Ideas:

- Word guessing game
- Rock, paper, scissors
- Simple adventure game

Game Developer!

You've built your first complete interactive game!

Figure 6.28 — Number Guessing Game Challenge — Capstone Programming Project. This comprehensive game development project integrates all Chapter 6 concepts—loops, conditionals, input validation, error handling, and user interaction—into an engaging guessing game. Students apply their complete programming skillset to create a fun, interactive experience with feedback and celebration features.

Shape Artist Challenge

Build your own geometric art creation tool

Complete Implementation

```

1 import turtle
2
3 # Geometric drawing functions
4 def draw_polygon(sides, size):
5     angle = 360 / sides
6     for i in range(sides):
7         artist.forward(size)
8         artist.right(angle)
9
10 # Main application loop
11 while True:
12     shape = input("Shape: ")
13     if shape == "square":
14         draw_polygon(4, size)
15     elif shape == "triangle":

```

Mathematical Programming:

- Square: $360^\circ/4 = 90^\circ$ • Triangle: $360^\circ/3 = 120^\circ$
- Pentagon: $360^\circ/5 = 72^\circ$ • Any polygon: $360^\circ/n$

Advanced Skills:

- Functions with parameters • Mathematical calculations
- Modular programming • Interactive art creation

Creative Application

Shape Artist Studio



Square



Triangle



Pentagon

User Interaction:

Shape: pentagon
Size: 60
Color: green
Drawing green pentagon...

Application Features:

- Multiple geometric shapes
- Custom size and color control
- Mathematical precision

Your Challenge:

Build this complete creative tool!

Advanced Concepts

Mathematical Foundations:

- Polygon angle formula: $360^\circ/n$
- Interior angles: $180^\circ(n-2)/n$
- Parametric function design

Math + Programming = Art!

Creative Extensions:

- Star shapes (skip vertices)
- Spiral polygons
- Fractal patterns
- 3D perspective effects

Professional Applications:

- CAD software development
- Educational geometry tools
- Digital art applications

Math + Art Master!

You're combining mathematics, programming, and creativity!

Figure 6.29 — Shape Artist Challenge — Creative Programming Application. This comprehensive creative programming project challenges students to build a complete shape—drawing application that combines user input, conditional logic, geometric calculations, and artistic output. Students create a professional—quality tool that demonstrates mastery of functions, loops, and interactive programming.

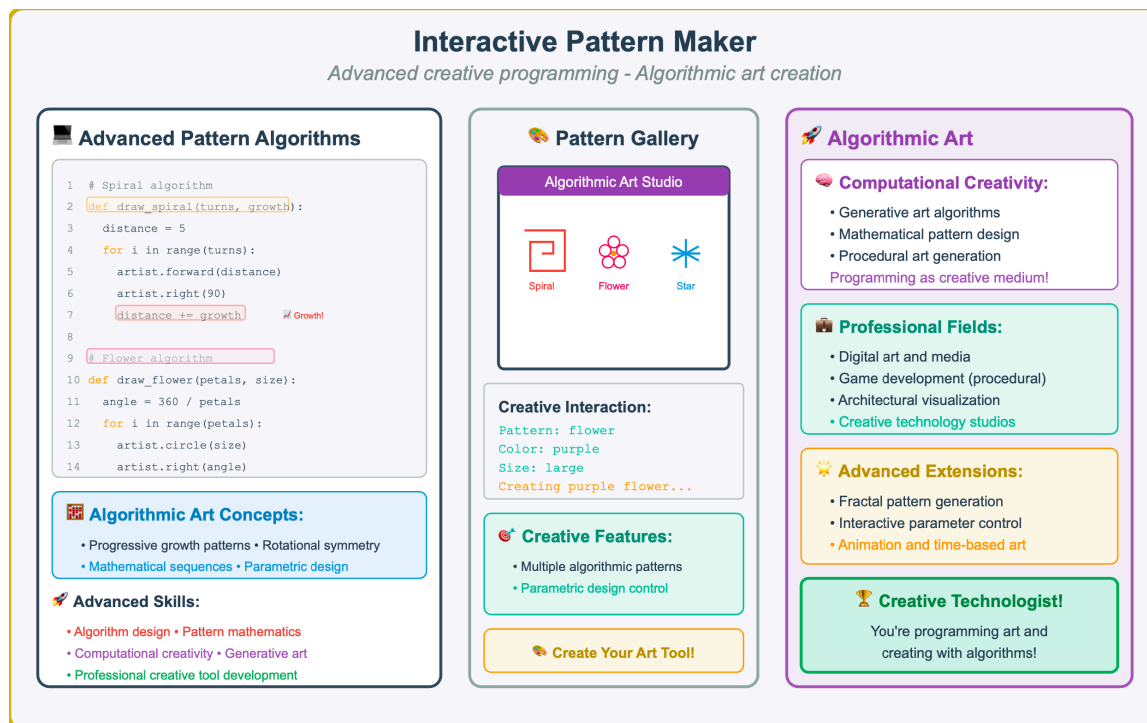


Figure 6.30 — Interactive Pattern Maker — Advanced Creative Programming Challenge. This sophisticated creative programming project challenges students to build an interactive pattern generation tool that combines advanced loop techniques, mathematical concepts, and artistic design. Students create complex algorithmic art while mastering advanced programming patterns and creative problem—solving.